15 Critical Frontend Interview Concepts (Quick Reference)

1. Event Loop (Very Common Question)

What is it? How JavaScript manages asynchronous operations and executes code in the correct order.

The Process: 1. **Call Stack** - Executes synchronous code 2. **Web APIs** - Handle setTimeout, fetch, etc. 3. **Callback Queue** - Holds completed callbacks 4. **Microtask Queue** - Promises, async/await (higher priority) 5. Event loop checks if stack is empty, then moves callbacks to stack

Example:

```
console.log('Start');

setTimeout(() => {
  console.log('setTimeout');
}, 0);

Promise.resolve()
  .then(() => console.log('Promise'));

console.log('End');

// Output:
// Start
// End
// Promise (microtask - higher priority than callback queue)
// setTimeout (callback queue)
```

Why It Matters: Essential for understanding async behavior, debugging timing issues.

---

2. Closures (Very Common Question)

What is it? A function that has access to variables from its outer scope even after the outer function has returned.

Example:

```
function outer(x) {
  return function inner(y) {
    return x + y; // inner can access x even after outer returns
  };
}
```

```
const add5 = outer(5);
console.log(add5(3)); // 8

// Practical: Data encapsulation
function createCounter() {
  let count = 0;
  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 2
console.log(counter.count); // undefined (private!)
```

Why It Matters: Foundation for understanding scope, data privacy, callbacks, React hooks.

---

3. Hoisting

What is it? JavaScript's behavior of moving declarations to the top of their scope.

var (hoisted + initialized with undefined):

```
console.log(x); // undefined (not an error!)
var x = 5;
console.log(x); // 5

// Interpreted as:
// var x;
// console.log(x);
// x = 5;
```

let/const (hoisted but not initialized - Temporal Dead Zone):

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 5;

// They're in the TDZ from start of block until the line they're declared
```

Functions (fully hoisted):

```
console.log(add(2, 3)); // 5 - works fine!

function add(a, b) {
  return a + b;
}

// But function expressions are not:
console.log(multiply(2, 3)); // TypeError: multiply is not a function
const multiply = (a, b) => a * b;
```

Why It Matters: Avoid bugs, understand variable declarations properly.

---

4. Debouncing & Throttling (Very Common Question)

Debouncing: Execute function only after user stops triggering (e.g., search input, resize)

```
function debounce(func, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => func.apply(this, args), delay);
  };
}

// Usage: Search input
const handleSearch = debounce((query) => {
  console.log('Searching for:', query);
}, 500);

input.addEventListener('input', (e) => handleSearch(e.target.value));
// Makes API call only 500ms after user stops typing
```

Throttling: Execute function at most once per interval (e.g., scroll, window resize)

```
function throttle(func, interval) {
  let lastRun = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastRun >= interval) {
      func.apply(this, args);
      lastRun = now;
    }
  };
}
```

```
// Usage: Scroll event
const handleScroll = throttle(() => {
  console.log('Scrolling...');
}, 1000);

window.addEventListener('scroll', handleScroll);
// Executes at most once per 1000ms
```

Key Difference: - **Debounce:** Wait until user stops → execute once - **Throttle:** Execute at regular intervals

Why It Matters: Performance optimization, interview favorite.

---

5. This Keyword

This depends on how a function is called:

```
// 1. Method call -> this = object
const obj = {
  name: 'John',
  greet: function() {
    console.log(this.name);
  }
};
obj.greet(); // 'John'

// 2. Regular function call -> this = undefined (strict mode) or window (non-strict)
function speak() {
  console.log(this);
}
speak(); // undefined (strict mode)

// 3. Constructor call -> this = new object
function Person(name) {
  this.name = name;
}
const p = new Person('Alice'); // this = p

// 4. Explicit binding with call/apply/bind
function introduce(greeting) {
  console.log(greeting + ', ' + this.name);
}
introduce.call(obj, 'Hello'); // 'Hello, John'
introduce.apply(obj, ['Hi']); // 'Hi, John'
const boundIntro = introduce.bind(obj, 'Hey');
```

```
boundIntro(); // 'Hey, John'

// 5. Arrow functions -> lexical this (from surrounding scope)
const person = {
  name: 'Bob',
  greet: () => {
    console.log(this); // this from outer scope, not person!
  }
};
```

Why It Matters: Common bugs, crucial for understanding JavaScript context.

---

6. Prototypes & Prototypal Inheritance

What is it? JavaScript uses prototypes for inheritance (not classical classes).

```
// Prototype chain
const parent = {
  greet() {
    return `Hello, ${this.name}`;
  }
};

const child = Object.create(parent);
child.name = 'Alice';

console.log(child.greet()); // 'Hello, Alice'
// child doesn't have greet, but finds it in parent (prototype)

// Constructor function pattern
function Vehicle(type) {
  this.type = type;
}

Vehicle.prototype.describe = function() {
  return `This is a ${this.type}`;
};

const car = new Vehicle('car');
console.log(car.describe()); // 'This is a car'

// Check prototype chain
console.log(car instanceof Vehicle); // true
console.log(Object.getPrototypeOf(car) === Vehicle.prototype); // true
```

Why It Matters: Understanding object inheritance, prototype pollution vul-

nerabilities.

---

7. Promises & Async/Await

Promises:

```
// Promise states: pending, fulfilled, rejected
const promise = new Promise((resolve, reject) => {
  if (condition) {
    resolve('Success');
  } else {
    reject('Error');
  }
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error))
  .finally(() => console.log('Done'));

// Promise.all - wait for all promises
Promise.all([promise1, promise2, promise3])
  .then(results => console.log(results))
  .catch(error => console.log('One failed:', error));

// Promise.race - fastest promise wins
Promise.race([promise1, promise2])
  .then(result => console.log(result));
```

Async/Await (syntactic sugar over Promises):

```
async function fetchUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);
    if (!response.ok) throw new Error('Failed to fetch');
    const data = await response.json();
    return data;
  } catch (error) {
    console.log('Error:', error);
  }
}

// Parallel requests
async function fetchMultiple() {
  const [user, posts, comments] = await Promise.all([
    fetchUser(1),
```

```
    fetchPosts(1),
    fetchComments(1)
  ]);
}
```

Why It Matters: Essential for API calls, asynchronous operations.

---

8. React Hooks Deep Dive

useState:

```
const [state, setState] = useState(initialValue);

// Batching (automatic in React 18)
// Multiple setState calls are batched into one render

const [count, setCount] = useState(0);
const handleClick = () => {
  setCount(count + 1);
  setCount(count + 1);
  // Still renders once with count = 1, not 2
};
```

useEffect:

```
useEffect(() => {
  // Runs after render (side effects)

  return () => {
    // Cleanup function (runs before component unmounts or before next effect)
  };
}, [dependencies]);

// Dependency array controls when effect runs:
// [] - runs once after initial render
// [dep1, dep2] - runs when dependencies change
// no array - runs after every render (avoid!)

// Common pattern: fetch data
useEffect(() => {
  let isMounted = true;

  fetch('/api/data')
    .then(res => res.json())
    .then(data => {
      if (isMounted) setData(data); // Prevent state update on unmounted component
```

```
    });

  return () => {
    isMounted = false; // Cleanup
  };
}, []);
```

useCallback & useMemo:

```
// useCallback - memoize function (only recreate if deps change)
const memoizedCallback = useCallback(() => {
  doSomething(a, b);
}, [a, b]); // Only recreates if a or b changes

// useMemo - memoize expensive computation
const memoizedValue = useMemo(() => {
  return expensiveComputation(a, b);
}, [a, b]);

//  Warning: Don't overuse! Premature optimization is bad.
// Only use if you have performance issues.
```

useRef:

```
const inputRef = useRef(null);

// Ref persists across renders
// Not part of render cycle - doesn't cause re-render when changed

const focusInput = () => {
  inputRef.current.focus();
};

return <input ref={inputRef} />;
```

Why It Matters: Core of modern React development.

---

9. Context API & State Management

When to use Context vs Props:

```
// Prop drilling (bad - passing through many intermediate components)
<GrandParent>
  <Parent userRole={userRole}>
    <Child userRole={userRole}>
      <GrandChild userRole={userRole} />
    </Child>
```

```
  </Parent>
</GrandParent>

// Context API solution
const UserContext = createContext();

function App() {
  const [userRole, setUserRole] = useState('admin');

  return (
    <UserContext.Provider value={{ userRole, setUserRole }}>
      <GrandParent />
    </UserContext.Provider>
  );
}

function GrandChild() {
  const { userRole } = useContext(UserContext);
  return <div>{userRole}</div>;
}
```

When to use Redux/Zustand: - Large app with complex state logic - Need middleware/time-travel debugging - Multiple independent state slices

Why It Matters: Choosing right state management impacts app architecture.

---

10. Virtual DOM & React Reconciliation

What is Virtual DOM? - In-memory representation of real DOM - React creates new VDOM tree after each render - Compares (diffs) with previous VDOM - Only updates changed parts in real DOM

Example:

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      {/* This <p> doesn't change, won't be updated in real DOM */}
      <p>Static text</p>
    </div>
  );
}
```

```
// Only the <p> with count updates in real DOM
```

Key prop importance:

```
//  Bad - list items might re-render unnecessarily
{items.map((item, index) => <Item key={index} data={item} />)}

//  Good - unique, stable keys
{items.map(item => <Item key={item.id} data={item} />)}
```

Why It Matters: Understanding performance, why keys matter.

---

11. Flexbox & CSS Grid

Flexbox (1D layout - rows or columns):

```
.flex-container {
  display: flex;

  /* Main axis (horizontal in row direction) */
  justify-content: space-between; /* spread items */

  /* Cross axis (vertical in row direction) */
  align-items: center;

  /* Wrapping */
  flex-wrap: wrap;

  gap: 1rem; /* space between items */
}

.flex-item {
  flex: 1; /* grow equally */
  flex-basis: 200px; /* base size */
}
```

CSS Grid (2D layout - rows and columns):

```
.grid-container {
  display: grid;

  /* Define columns */
  grid-template-columns: 1fr 2fr 1fr;

  /* Or auto-responsive */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

```
  grid-template-rows: 100px 200px;
  gap: 2rem;
}

.grid-item {
  grid-column: 1 / 3; /* span 2 columns */
  grid-row: 2; /* row 2 */
}
```

When to use which: - **Flexbox:** Single row/column layouts, simple component layouts - **Grid:** Complex 2D layouts, page layouts, dashboards

Why It Matters: Essential for responsive design, very common in interviews.

---

12. Responsive Design & Mobile-First

Mobile-First Approach:

```
/* Mobile styles (default, smaller screens) */
.container {
  font-size: 14px;
  width: 100%;
  flex-direction: column;
}

/* Tablet and above */
@media (min-width: 768px) {
  .container {
    font-size: 16px;
    width: 90%;
    flex-direction: row;
  }
}

/* Desktop and above */
@media (min-width: 1024px) {
  .container {
    font-size: 18px;
    width: 80%;
    max-width: 1200px;
  }
}
```

Viewport Meta Tag (REQUIRED):

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Why It Matters: Most users browse on mobile, essential for modern web.

---

13. Performance Optimization

Key Metrics: - **LCP** (Largest Contentful Paint) - when main content visible - **FID** (First Input Delay) - responsiveness to user input - **CLS** (Cumulative Layout Shift) - visual stability

Optimization Strategies:

```
// 1. Code splitting (load only what's needed)
const HeavyComponent = React.lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <Suspense fallback={<Loading />}>
      <HeavyComponent />
    </Suspense>
  );
}

// 2. Lazy loading images
<img loading="lazy" src="..." />

// Or with Intersection Observer
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
    }
  });
});

document.querySelectorAll('[data-src]').forEach(img => observer.observe(img));

// 3. Memoization (avoid recalculation)
const memoizedResult = useMemo(() => expensiveCalculation(), [deps]);

// 4. Request debouncing
const debouncedSearch = debounce((query) => {
  fetch(`/search?q=${query}`);
}, 500);
```

Why It Matters: Users expect fast websites, performance is user experience.

14. CORS & HTTP Basics

HTTP Methods: - **GET** - Retrieve data (safe, idempotent) - **POST** - Create data - **PUT** - Replace entire resource - **PATCH** - Partial update - **DELETE** - Remove data

Status Codes:

```
2xx - Success
  200 - OK
  201 - Created
  204 - No Content

3xx - Redirection
  301 - Moved Permanently
  304 - Not Modified

4xx - Client Error
  400 - Bad Request
  401 - Unauthorized
  403 - Forbidden
  404 - Not Found

5xx - Server Error
  500 - Internal Server Error
  503 - Service Unavailable
```

CORS (Cross-Origin Resource Sharing):

```javascript
// Browser blocks requests to different origin by default
// Solution 1: Server sends CORS headers
// Response headers: Access-Control-Allow-Origin: *

// Solution 2: Use JSONP (deprecated)

// Solution 3: Proxy your API in development
// In React: "proxy": "http://localhost:5000" in package.json

// Solution 4: Use credentials
fetch('https://api.example.com/data', {
  credentials: 'include', // Send cookies
  headers: {
    'Content-Type': 'application/json'
  }
});
```

Why It Matters: Working with APIs is core frontend skill.

---

15. Error Handling & Debugging

Error Boundaries (Catch React component errors):

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    console.log('Error:', error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong</h1>;
    }
    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

Try-Catch for Async:

```
async function fetchData() {
  try {
    const response = await fetch('/api/data');
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Fetch error:', error);
    // Handle error (show toast, retry, etc.)
  }
}
```

Debugging Tips: - Use Chrome DevTools (breakpoints, watch expressions) - console.log / console.table for data inspection - Network tab for API issues - React DevTools extension - Performance profiler for bottlenecks

Why It Matters: Good error handling makes apps robust and user-friendly.

Quick Study Tips

1. **Understand the "why"** not just the "how"
2. **Practice writing code** for each concept
3. **Explain out loud** - hearing yourself helps memory
4. **Build projects** that use these concepts
5. **Review periodically** - spaced repetition

Interview Pro Tip: When asked a concept, start with a simple explanation, then offer to go deeper. Example: > "Closures are functions that remember variables from their outer scope. For instance…" (explain with code). "Would you like me to explain a real-world use case like data encapsulation?"

Good luck!